

Harshit→

```
# You can add your own Logical error catagorization in Taxonomy_Text

TAXONOMY_TEXT = """
<taxonomy>
  <category label="A">
    <title>Boundary & Indexing</title>
    <description>Errors in loop bounds or indexing (off-by-one, skipped or extra
element).</description>
  </category>
  <category label="B">
    <title>Conditional / Boolean</title>
    <description>Incorrect condition, wrong boolean operator, reversed logic, unreachable
branch.</description>
  </category>
  <category label="C">
    <title>State Management</title>
    <description>Incorrect handling of variables across execution (not resetting counters,
stale state).</description>
  </category>
  <category label="D">
    <title>Algorithmic Strategy</title>
    <description>Fundamental flaw in approach or data structure; logic does not solve the
task.</description>
  </category>
  <category label="E">
    <title>Edge Case Handling</title>
    <description>Fails only on boundary inputs (empty input, single element, zero, negative
values).</description>
  </category>
  <category label="F">
    <title>Specification Misunderstanding</title>
    <description>Code does not follow the problem statement, output format, or required
function behavior.</description>
  </category>
  <category label="NONE">
    <title>No Error</title>
    <description>No logical error present.</description>
  </category>
</taxonomy>
"""

# =====
# PROMPTS
# =====
PROMPT_ITER1_SINGLE = f"""
<role>Expert Python Programming Assistant</role>
```

<task>
Identify the SINGLE most dominant logical error in the provided Python code based on the taxonomy below.

</task>

{TAXONOMY_TEXT}

<instructions>

1. Analyze the code inside the <code_to_analyze> tags.
2. If multiple errors exist, select the one that most fundamentally causes the code to fail.
3. If no logical error exists, you must return NONE.

</instructions>

<output_rules>

- Output ONLY the single character label (A, B, C, D, E, F) or NONE.
- Do NOT include the category title, explanations, or any punctuation.
- STRICT: Any text other than the label will be considered a failure.

</output_rules>

<code_to_analyze>

{{code_input}}

</code_to_analyze>

"""

PROMPT_ITER_MULTI = f"""

<role>Expert Python Programming Assistant</role>

<task>

Identify ALL applicable logical error types in the provided Python code based on the taxonomy below.

</task>

{TAXONOMY_TEXT}

<instructions>

1. Evaluate the code inside the <code_to_analyze> tags against every category in the taxonomy.
2. Select all labels that apply to the logic of the code.
3. If no logical errors are found, use the label: NONE.

</instructions>

<output_rules>

- Output labels as a comma-separated list (e.g., A,C,E).
- If NONE is chosen, it must be the ONLY label in the output.
- Do NOT include spaces, titles, or descriptions.
- STRICT: No conversational filler or explanations.

</output_rules>

<code_to_analyze>

{{code_input}}

</code_to_analyze>

"""

Yuvraj→

```
# You can add your own Logical error categorization in Taxonomy_Text
```

```
TAXONOMY_TEXT = ""
```

```
<taxonomy>
  <category label="A">
    <title>Input</title>
    <description>Failing to receive all input values or using the incorrect data type for
variables.</description>
  </category>
  <category label="B">
    <title>Output</title>
    <description>Non-compliance with required formats or outputting incorrect string
literals.</description>
  </category>
  <category label="C">
    <title>Variable</title>
    <description>Storing incorrect values or specifying the wrong data type for a
variable.</description>
  </category>
  <category label="D">
    <title>Computation</title>
    <description>Calculating with incorrect values or using the wrong
operations.</description>
  </category>
  <category label="E">
    <title>Condition</title>
    <description>Incorrect or insufficient conditional operations in declarations.</description>
  </category>
  <category label="F">
    <title>Branching</title>
    <description>Incorrect program flow (e.g., wrong break placement or if-if instead of
if-else).</description>
  </category>
  <category label="G">
    <title>Loop</title>
    <description>Incorrect conditions or variables used in loop declarations.</description>
  </category>
  <category label="H">
    <title>Array/String</title>
    <description>Incorrect initialization or referencing an incorrect index.</description>
  </category>
  <category label="I">
    <title>Function</title>
    <description>Incorrectly defined parameters/return values or wrong arguments in
calls.</description>
  </category>
```

```

    <category label="J">
        <title>Conceptual</title>
        <description>Solving the wrong problem or missing necessary loops/conditions
entirely.</description>
    </category>
    <category label="N">
        <title>No Error</title>
        <description>No logical error present.</description>
    </category>
</taxonomy>
"""

```

```

# =====

```

```

# PROMPTS

```

```

# =====

```

```

PROMPT_ITER1_SINGLE = f"""

```

```

<role>Expert Python Programming Assistant</role>

```

```

<task>

```

Identify the SINGLE most dominant logical error in the provided Python code based on the taxonomy below.

```

</task>

```

```

{TAXONOMY_TEXT}

```

```

<instructions>

```

1. Analyze the code inside the <code_to_analyze> tags.
2. If multiple errors exist, select the one that most fundamentally causes the code to fail.
3. If no logical error exists, you must return NONE.

```

</instructions>

```

```

<output_rules>

```

- Output ONLY the single character label (A, B, C, D, E, F) or NONE.
- Do NOT include the category title, explanations, or any punctuation.
- STRICT: Any text other than the label will be considered a failure.

```

</output_rules>

```

```

<code_to_analyze>

```

```

{{code_input}}

```

```

</code_to_analyze>

```

```

"""

```

```

PROMPT_ITER_MULTI = f"""

```

```

<role>Expert Python Programming Assistant</role>

```

```

<task>

```

Identify ALL applicable logical error types in the provided Python code based on the taxonomy below.

```

</task>

```

{TAXONOMY_TEXT}

<instructions>

1. Evaluate the code inside the <code_to_analyze> tags against every category in the taxonomy.
2. Select all labels that apply to the logic of the code.
3. If no logical errors are found, use the label: NONE.

</instructions>

<output_rules>

- Output labels as a comma-separated list (e.g., A,C,E).
- If NONE is chosen, it must be the ONLY label in the output.
- Do NOT include spaces, titles, or descriptions.
- STRICT: No conversational filler or explanations.

</output_rules>

<code_to_analyze>

{{code_input}}

</code_to_analyze>

""""

Tanishi →

```
# You can add your own Logical error categorization in Taxonomy_Text

TAXONOMY_TEXT = ""
<taxonomy>
  <category label="A">
    <title>Loop Condition</title>
    <description>Incorrect loops in the for/while condition.</description>
  </category>

  <category label="B">
    <title>Condition Branch</title>
    <description>Incorrect expression in the if condition.</description>
  </category>

  <category label="C">
    <title>Statement Integrity</title>
    <description>Statement lacks a required part of the logical structure.</description>
  </category>

  <category label="D">
    <title>Output / Input Format</title>
    <description>Incorrect cin/cout or input-output statement.</description>
  </category>

  <category label="E">
    <title>Variable Initialization</title>
    <description>Incorrect declaration or initialization of variables.</description>
  </category>

  <category label="F">
    <title>Data Type</title>
    <description>Incorrect data type used.</description>
  </category>

  <category label="G">
    <title>Computation</title>
    <description>Incorrect basic math operators or calculations.</description>
  </category>

  <category label="NONE">
    <title>No Error</title>
    <description>No logical error present.</description>
  </category>
</taxonomy>

""
```

```
# =====
```

```
# PROMPTS
```

```
# =====
```

```
PROMPT_ITER1_SINGLE = f"""
```

```
<role>Expert Python Programming Assistant</role>
```

```
<task>
```

```
Identify the SINGLE most dominant logical error in the provided Python code based on the taxonomy below.
```

```
</task>
```

```
{TAXONOMY_TEXT}
```

```
<instructions>
```

1. Analyze the code inside the <code_to_analyze> tags.
2. If multiple errors exist, select the one that most fundamentally causes the code to fail.
3. If no logical error exists, you must return NONE.

```
</instructions>
```

```
<output_rules>
```

- Output ONLY the single character label (A, B, C, D, E, F) or NONE.
- Do NOT include the category title, explanations, or any punctuation.
- STRICT: Any text other than the label will be considered a failure.

```
</output_rules>
```

```
<code_to_analyze>
```

```
{{code_input}}
```

```
</code_to_analyze>
```

```
"""
```

```
PROMPT_ITER_MULTI = f"""
```

```
<role>Expert Python Programming Assistant</role>
```

```
<task>
```

```
Identify ALL applicable logical error types in the provided Python code based on the taxonomy below.
```

```
</task>
```

```
{TAXONOMY_TEXT}
```

```
<instructions>
```

1. Evaluate the code inside the <code_to_analyze> tags against every category in the taxonomy.
2. Select all labels that apply to the logic of the code.
3. If no logical errors are found, use the label: NONE.

```
</instructions>
```

```
<output_rules>
```

- Output labels as a comma-separated list (e.g., A,C,E).

- If NONE is chosen, it must be the ONLY label in the output.
 - Do NOT include spaces, titles, or descriptions.
 - STRICT: No conversational filler or explanations.
- </output_rules>

<code_to_analyze>
{{code_input}}
</code_to_analyze>
""